

ARCHITECTURE BASELINE

ProposalJa

Auditoria Completa do Ambiente Tecnico

Stack, Estrutura, DB, Auth, RLS, Fluxo, PDF, Deps, Scripts, Riscos, Plano

2026-08-07 | Protocolo Rule 48 | Audit v1.0

Índice

1. Stack Tecnológico	4
1.1 Core	4
1.2 DevOps e Testes	4
2. Estrutura do Projecto	5
3. Banco de Dados	5
3.1 Tabelas do Schema Public	6
3.2 Enums	6
3.3 Funcoes PostgreSQL (RPCs)	7
4. Autenticacao e Autorizacao	7
4.1 Fluxo Auth	7
4.2 Modelo de Autorizacao	7
4.3 Gap Identificado	8
5. Row Level Security (RLS)	8
5.1 Politicas por Tabela	8
5.2 Padrao de Funcoes Auxiliares	9
5.3 Estado da Migration do Blueprint Engine	9
6. Fluxo Actual de Propostas	9
6.1 Fluxo Simples (Producao)	9
6.2 Fluxo Avancado (Blueprint Engine)	9
6.3 Fluxo AI (Doc A - Proposta Comercial)	10
7. Sistema PDF	10
7.1 Arquitectura PDF	10
7.2 Theme System	10
7.3 Problemas Conhecidos	11
7.4 Strategia de Migracao Aprovada	11
8. Dependencias	11
8.1 Dependencias Criticas	11
8.2 Dependencias em Falta	12
9. Scripts Disponiveis	12

10. Pontos Criticos	12
10.1 Conectividade DB	12
10.2 Migration do Blueprint Engine	12
10.3 Tipos Supabase Auto-Generated	13
10.4 Separacao de Modulos	13
11. Riscos	13
12. Plano de Implementacao	13
Fase 1: Fundacao DB (Prioridade Maxima)	14
Fase 2: Document Model + Question Engine	14
Fase 3: AI Integration (por seccao)	14
Fase 4: HTML/CSS Renderer (Premium PDFs)	14
Fase 5: Brand Profiles + Design System	14
Fase 6: Revisao + Exportacao	15
Fase 7: Auditoria + Validacao Final	15

1. Stack Tecnológico

O ProposalJa é uma aplicação Single-Page Application (SPA) construída sobre um stack moderno de frontend com backend-as-a-service. Não existe um servidor de aplicação tradicional; toda a lógica de negócio reside no cliente React com chamadas directas ao Supabase via PostgREST API. A escolha deste stack permite desenvolvimento rápido e custos operacionais reduzidos, mas introduz restrições significativas em termos de processamento server-side e geração de PDFs premium.

1.1 Core

Camada	Tecnologia	Versão	Notas
Build Tool	Vite	5.4.19	SWC plugin para React
Framework UI	React	18.3.1	SPA, sem SSR
Linguagem	TypeScript	5.8.3	Strict mode
Routing	React Router DOM	6.30.1	BrowserRouter (hash-free)
State / Data	TanStack React Query	5.83.0	Server state cache
Forms	React Hook Form + Zod	7.61.1 / 3.25	Validação type-safe
UI Components	Radix UI + shadcn/ui	multiple	30+ componentes
CSS	Tailwind CSS	3.4.17	JIT, @apply mínimo
Backend / DB	Supabase (PostgreSQL)	-	PostgREST + RLS + Auth
Client DB	Supabase JS	2.104.1	Auto-refresh token
Analytics	PostHog	1.236.7	Identify + reset events
Temas	next-themes	0.3.0	Light/dark system

1.2 DevOps e Testes

Ferramenta	Versão	Uso
Vitest	3.2.4	Unit tests (test/test:watch)
Testing Library	16.0.0	Component tests + jsdom
Playwright	1.58.2	E2E tests (devDep, ainda sem suites)
ESLint	9.32.0	Lint estático
lovable-tagger	1.1.13	Component tagging (dev mode)
pg	8.22.0	Direct PostgreSQL (scripts de reparação)

Ponto notavel: O projecto usa Vite (Nao Next.js). Isto significa que nao existem API routes ou server-side rendering. Toda a logica AI corre via Supabase Edge Functions (invocadas por supabase.functions.invoke). O unico backend disponivel e o PostgREST do Supabase + Edge Functions + PostgreSQL RPCs.

2. Estrutura do Projecto

O projecto segue uma organizacao flat baseada em dominios funcionais. A pasta src/ contem todas as fontes da aplicacao, divididas em pages/, components/, services/, hooks/, lib/, e types/. A separacao entre propostas simples (fluxo actual) e propostas avancadas (Blueprint Engine) e feita pela existencia de pages/advanced/ e services/advancedProposalService.ts.

Pasta	Funcao	Ficheiros-chave
src/pages/	Paginas da SPA (14 ficheiros)	ResumoProposta.tsx, CriarProposta.tsx, Dashboard.tsx
src/pages/advanced/	Propostas avancadas (Blueprint)	NovaPropostaAvancada.tsx, PreencherProposta.tsx
src/pages/admin/	Painel de administracao	TenantsTab.tsx, UsersTab.tsx, MetricsTab.tsx, AuditTab.tsx
src/components/ui/	shadcn/ui (50+ componentes)	button.tsx, dialog.tsx, table.tsx, etc.
src/components/org/	Organizacao (membros, convites)	MemberList.tsx, InviteModal.tsx, RoleBadge.tsx
src/services/	Camada de servicos (10 ficheiros)	propostaService.ts, advancedProposalService.ts
src/lib/pdf/	Sistema PDF (11 ficheiros)	registry.ts, shared.ts, classic.ts, modern.ts, etc.
src/hooks/	Custom hooks (4 ficheiros)	useAuth.tsx, useOrganization.ts, useActivityTracker.tsx
src/types/	TypeScript types (3 ficheiros)	index.ts, advancedProposal.ts, admin.ts
src/integrations/	Supabase client + types gerados	client.ts, types.ts (auto-generated)
supabase/migrations/	Migrations SQL (30 ficheiros)	rls_org_scoped.sql, blueprint_engine.sql

Arquitectura de modulos: O fluxo de propostas simples (CriarProposta, ResumoProposta, GerarPropostaA) e completamente independente do fluxo avancado (NovaPropostaAvancada, PreencherProposta). Partilham apenas o Supabase client e a camada de autenticacao. Esta separacao modular e critica para a estrategia de implementacao: o fluxo simples permanece intocado enquanto o avancado evolui.

3. Banco de Dados

O Supabase fornece uma instancia PostgreSQL gerida com PostgREST. O schema public contem 21 tabelas cobrindo propostas, clientes, facturas, catalogo, organizacoes, AI, e o novo Blueprint Engine. A evolucao do schema foi rastreada ao longo de 30 migrations incrementais, desde o schema single-tenant inicial ate ao actual schema multi-tenant com RLS org-scoped.

3.1 Tabelas do Schema Public

#	Tabela	Tipo	RLS	Notas
1	profiles	Dados do utilizador	Sim	Extends auth.users
2	organizations	Tenant principal	Sim	Plano, limites, status
3	organization_members	Pertence org/user	Sim	Roles: owner/admin/member
4	organization_invitations	Convites pendentes	Sim	Token + expiracao
5	clients	Clientes por org	Sim	Unique email per org
6	proposals	Propostas simples	Sim	Snapshot do cliente, IVA, desconto
7	proposal_items	Items da proposta	Sim (via parent)	Cascade delete
8	invoices	Facturas	Sim	Conversion from proposal
9	invoice_items	Items da factura	Sim (via parent)	FK to invoices
10	catalog_items	Catalogo de servicos	Sim	Unique name per org
11	subscriptions	Assinaturas/plano	Sim	Per-user, per-org limits
12	proposta_ai	Geracao AI (Doc A)	Sim	Input/output/edited JSON
13	business_categories	Categorias de negocio	Nao (global)	Blueprint Engine
14	proposal_blueprints	Modelos de proposta	Nao (global)	Versionados por categoria
15	proposal_sections	Secoos do blueprint	Nao (global)	Content rules JSONB
16	section_questions	Perguntas por seccao	Nao (global)	Visibility rules JSONB
17	company_brand_profiles	Identidade visual	Sim (via org)	Cores, fonte, estilo
18	advanced_proposals	Propostas avancadas	Sim	Status machine 5 estados
19	proposal_section_answers	Respostas + AI	Sim (via parent)	Upsert on section_id
20	platform_audit_log	Auditoria global	Admin only	Login, export, RLS change
21	tenant_metrics	Metricas por tenant	Admin only	Contadores diarios

3.2 Enums

Enum	Valores
proposal_status	rascunho, enviada, aceite, rejeitada
invoice_status	rascunho, enviada, paga, vencida, cancelada
org_role	owner, admin, member
org_status	active, suspended, trial

Enum	Valores
plan_type	free, pro, enterprise
desconto_tipo	percentual, fixo
visual_style	corporate, premium, minimal, technical
content_status	pendente, gerando, gerado, editando, revisado, erro

3.3 Funcoes PostgreSQL (RPCs)

O sistema inclui funcoes SECURITY DEFINER que servem como pontes seguras entre o contexto `auth.uid()` e as operacoes de negocio. As funcoes criticas sao: `user_belongs_to_org(p_org_id)` que verifica se o utilizador actual e membro de uma organizacao especifica; `user_role_in_org(p_org_id)` que retorna o role do utilizador numa org; `has_org_role_min(p_uid, p_role)` que verifica se o utilizador tem um role minimo numa org qualquer; `has_role(p_uid, p_role)` que verifica roles ao nivel da plataforma (ex: admin). Adicionalmente, existem RPCs para metricas de plataforma, verificacao de limites de plano, e funcoes de trigger para auto-numeracao de propostas e facturas. Todas as funcoes SECURITY DEFINER usam `SET search_path = public` para evitar injection de `search_path`.

4. Autenticacao e Autorizacao

A autenticacao e inteiramente delegada ao Supabase Auth, com o cliente React a gerir a sessao via `localStorage`. O `AuthProvider` (`useAuth.tsx`) e o contexto central que expoe `user`, `session`, `organization`, `orgRole`, `memberships`, e funcoes auxiliares como `hasOrgRoleMin()` e `setActiveOrganization()`.

4.1 Fluxo Auth

- Supabase Auth provider: email/password com confirmacao
- Sessao persistida em `localStorage` com `autoRefreshToken = true`
- `onAuthStateChange`: `SIGNED_IN` -> `PostHog identify`; `SIGNED_OUT` -> `PostHog reset + redirect`
- `ProtectedRoute`: verifica `user != null`, senao redirecciona para `/auth`
- Aceite de convite: rota `/invite/accept` com token-based verification

4.2 Modelo de Autorizacao

O sistema implementa um modelo de autorizacao em 3 camadas: (1) Plataforma, com um role "admin" que tem acesso global a todos os tenants e dados; (2) Organizacao, com roles owner/admin/member geridos pela tabela `organization_members`; (3) Dados, com RLS policies que verificam `user_belongs_to_org(organization_id)` para cada operacao. A funcao `hasOrgRoleMin()`

permite verificacao de role minimo (ex: member <= admin <= owner). Admins de plataforma contornam todas as policies de organizacao via has_role(auth.uid(), "admin").

4.3 Gap Identificado

Nao existe middleware server-side. Toda a verificacao de roles acontece no cliente (React) e nas RLS policies (PostgreSQL). Um utilizador com conhecimento tecnico pode modificar o cliente e contornar verificacoes de UI (ex: mostrar o botao de admin). Porem, as RLS policies protegem os dados de forma inviolavel no servidor. O risco e limitado a operacoes que dependem exclusivamente de logica cliente (ex: breadcrumbs, visibilidade de menus) e nao de operacoes de dados.

5. Row Level Security (RLS)

Todas as 21 tabelas com dados de negocio possuem RLS activo. As politicas foram evoluindo ao longo das migrations: da abordagem inicial baseada em owner_id simples, para o modelo actual org-scoped que usa a funcao user_belongs_to_org(p_org_id) para validar pertença a organizacao especifica de cada registo. As tabelas globais do Blueprint Engine (business_categories, proposal_blueprints, proposal_sections, section_questions) nao tem RLS pois sao dados de referencia partilhados por todos os tenants.

5.1 Politicas por Tabela

Tabela	Select	Insert	Update	Delete	Modelo
organizations	membro OR admin		owner/admin OR admin		Org-scoped
organization_members	membro OR admin	owner/admin	owner/admin	owner/admin	Org-scoped
clients	org OR owner	org OR owner	org OR owner	org OR owner	Dual (org+owner)
proposals	org OR owner	org+member	org OR owner	org+admin	Org-scoped + role
proposal_items	via parent	via parent	via parent	via parent	Subquery parent
invoices	org OR owner	org+member	org OR owner	org+admin	Org-scoped + role
invoice_items	via parent	via parent	via parent	via parent	Subquery parent
proposta_ai	org OR user	org OR user	org OR user	org OR user	Org-scoped + user
profiles	self OR org member		-	-	Self + org members
advanced_proposals	org-scoped	org-scoped	org-scoped	org-scoped	Org-scoped (novo)
proposal_section_answers	via parent	via parent	via parent	via parent	Subquery parent (novo)
company_brand_profiles	org-scoped	org-scoped	org-scoped	org-scoped	Org-scoped (novo)

5.2 Padrao de Funcoes Auxiliares

As RLS policies usam funcoes SECURITY DEFINER com SET search_path = public. Este padrao e critico para seguranca: sem SET search_path, um utilizador malicioso poderia criar funcoes com nomes que sobrepujam as funcoes do sistema e assim executar codigo arbitrario dentro do contexto SECURITY DEFINER. As funcoes-chave sao user_belongs_to_org(UUID) que faz EXISTS SELECT na tabela organization_members, e user_role_in_org(UUID) que retorna o enum org_role. A funcao has_role(auth.uid(), role) verifica se o utilizador e admin de plataforma consultando a tabela profiles.role diretamente.

5.3 Estado da Migration do Blueprint Engine

A migration 20260807000000 cria as 5 novas tabelas do Blueprint Engine, indexes, triggers, e RLS policies. Porem, a conectividade directa a base de dados nao esta disponivel neste ambiente (IPv6 ENETUNREACH), pelo que nao foi possivel verificar se esta migration ja foi aplicada ao Supabase remoto. O SQL foi validado sintacticamente e e idempotente (CREATE IF NOT EXISTS). A migration deve ser aplicada via Supabase Dashboard > SQL Editor ou Supabase CLI (supabase db push) antes de qualquer operacao nas novas tabelas.

6. Fluxo Actual de Propostas

O ProposalJa tem dois fluxos distintos de criacao de propostas: o fluxo simples (existente, estavel, usado em producao) e o fluxo avancado (Blueprint Engine, em implementacao). Ambos produzem PDFs, mas por caminhos diferentes.

6.1 Fluxo Simples (Producao)

O fluxo simples comeca na pagina CriarProposta.tsx, onde o utilizador selecciona um cliente, adiciona items ao catalogo ou manualmente, define desconto e IVA, e grava a proposta. A PropostaService.criarProposta() cria um snapshot do cliente, calcula totais via calculos.ts, insere a proposta e os items numa transacao logica (delete + re-insert em caso de edicao). A pagina ResumoProposta.tsx mostra a proposta completa com opcoes de editar, duplicar, gerar PDF, ou converter em factura. O PDF e gerado client-side via jsPDF com o Registry Pattern (6 templates: classic, modern, executive = free; sleek, sidebar, business = PRO).

6.2 Fluxo Avancado (Blueprint Engine)

O fluxo avancado comeca em NovaPropostaAvancada.tsx com a seleccao de categoria de negocio (business_categories) seguida da seleccao de blueprint (proposal_blueprints). Apos criacao do registo advanced_proposals, o utilizador navega para PreencherProposta.tsx que apresenta um wizard de seccoes com perguntas fixas (section_questions). As respostas sao gravadas em proposal_section_answers via upsert. O fluxo de geracao AI (por seccao) e de renderizacao premium (HTML/CSS + Playwright) ainda nao esta implementado - e o principal objectivo da implementacao que se segue a esta auditoria.

6.3 Fluxo AI (Doc A - Proposta Comercial)

Existe um terceiro fluxo que cruza os dois: a geracao AI de conteudo narrativo. Na pagina GerarPropostaIA.tsx, o utilizador preenche campos de contexto (problema, solucao, beneficios, etc.), escolhe o tom e o modelo AI, e invoca a Edge Function generate-proposal via propostaAiService.ts. O resultado e gravado em proposta_ai e pode ser exportado como PDF narrativo via narrativa.ts (PDF independente sem tabela de items). Este fluxo funciona independentemente do fluxo de propostas simples e nao partilha dados com o Blueprint Engine.

7. Sistema PDF

O sistema PDF e o componente mais complexo e mais criticado do ProposalJa. Usa jsPDF client-side com o Registry Pattern para gerar documentos PDF directamente no browser. O sistema compoe-se de 11 ficheiros TypeScript na pasta src/lib/pdf/, totalizando aproximadamente 980 chamadas doc.* distribuidas por 8 renderizadores.

7.1 Arquitectura PDF

Ficheiro	Funcao	doc.* calls	Notas
registry.ts	Registo central de templates	0	Map<string, TemplateEntry>
types.ts	Tipos (PdfTheme, TemplateEntry)	0	Interface definitions
themes.ts	6 temas visuais (PdfTheme)	0	Cores, fontes, spacing
shared.ts	Funcoes partilhadas	538	drawItemsTable, drawTotals, drawPaymentMethods, drawFooter
index.ts	Ponto de entrada unificado	0	gerarPDF() -> getTemplate().render()
classic.ts	Template Classico	45	Free, header colorido
modern.ts	Template Moderno	45	Free, centralizado
executive.ts	Template Executivo	50	Free, barra lateral accent
sleek.ts	Template Sleek	63	PRO, badges + stripe
sidebar.ts	Template Sidebar	71	PRO, barra lateral escura (quebra abstracao)
business.ts	Template Business	52	PRO, estilo factura premium
narrativa.ts	PDF narrativo AI	118	Ignora template param, output independente

7.2 Theme System

Cada template define um PdfTheme com 5 subsistemas: table (headerBg, columnRatios, theme), totals (position, showCard, totalHighlight), payment (position, style, title), footer (style, showBranding), e narrative (enabled, headingSize, bodySize, bulletStyle). As shared functions (drawItemsTable, drawTotals, etc.) leem de ctx.theme com defaults que replicam o comportamento

anterior a introducao do theme system. A funcao createContext() cria o PDFContext que e passada a todas as render functions.

7.3 Problemas Conhecidos

- **sidebar.ts quebra a abstracao:** Usa drawItemsTable() e drawTotals() de shared.ts mas reimplementa header e pagamento, ignorando o theme system parcialmente.
- **narrativa.ts ignora o template:** O parametro template e recebido mas nao usado; gera sempre o mesmo layout fixo sem suporte a cores/estilo da empresa.
- **jsPDF limitacoes:** Sem suporte nativo a CSS flexbox/grid; layout manual via coordenadas absolutas; sem kerning de fontes; sem suporte a imagens SVG; sem embed de fontes custom (limitado a Helvetica/Times/Courier).
- **Tamanho do bundle:** ~980 chamadas doc.* resultam num bundle PDF significativo. Lazy loading via dynamic import de jspdf-autotable mitiga parcialmente.

7.4 Strategia de Migracao Aprovada

Foi aprovada a migracao do sistema PDF premium para HTML/CSS + Playwright: (1) Templates free (classic, modern, executive) mantem jsPDF com melhorias incrementais; (2) Templates PRO (sleek, sidebar, business) e novos templates avancados usam HTML/CSS + Playwright para renderizacao no browser, com page.pdf() para output vectorial; (3) O narrativa.ts e completamente substituido pelo novo sistema HTML/CSS; (4) O Registry Pattern e mantido, mas o render function passa a gerar HTML em vez de usar jsPDF diretamente para templates PRO.

8. Dependencias

O projecto tem 34 dependencias de producao e 17 devDependencies. Abaixo estao as dependencias criticas para o sistema de propostas avancadas.

8.1 Dependencias Criticas

Pacote	Versao	Uso no Sistema Avancado	Risco
jspdf	4.2.1	PDF standard (free templates)	Manter, nao remover
jspdf-autotable	5.0.7	Tabelas nos PDFs	Manter
@supabase/supabase-js	2.104.1	Toda a comunicacao DB	Core dependency
react-router-dom	6.30.1	Navegacao SPA	Sem risco
zod	3.25.76	Validacao de forms	Usar para AI input validation
@playwright/test	1.58.2	E2E tests (devDep)	Precisa upgrade para browser PDF
pg	8.22.0	Scripts de reparacao DB	Somente scripts, nao runtime
recharts	2.15.4	Graficos no dashboard	Sem impacto no sistema avancado

8.2 Dependencias em Falta

- **playwright (runtime):** Necessario para HTML/CSS -> PDF premium. Actualmente apenas como devDep. Precisa ser adicionado como dependency de producao ou usar CDN bundle.
- **marked / markdown-it:** Necessario para converter markdown (output AI) em HTML para os PDFs premium.
- **html2canvas ou similar:** Apenas se for necessario fallback para navegadores sem suporte a `page.pdf()`.

9. Scripts Disponiveis

Tres scripts SQL de seguranca foram criados na sessao anterior para protecao contra danos na base de dados. Todos estao disponiveis em `/home/z/my-project/scripts/` e copiados para `/download/`.

Script	Linhas	Funcao
<code>proposaja_db_repair.sql</code>	~800	Reparacao idempotente: 16 seccoes (diagnosticos, enums, funcoes, tabelas, colunas, co
<code>proposaja_db_backup_restore.sql</code>	~120	Funcoes de restore point: <code>create_restore_point()</code> , <code>restore_from_point()</code> , <code>cleanup_restore_</code>
<code>proposaja_db_diagnostics.sql</code>	~300	25+ SELECT queries: inventario de schema, gaps RLS, dados orfaos, cross-tenant leaks

Alem destes, os npm scripts definidos em `package.json` sao: `dev` (vite), `build` (vite build), `build:dev` (vite build --mode development), `lint` (eslint), `preview` (vite preview), `test` (vitest run), e `test:watch` (vitest). Nao existem scripts de migracao, seed, ou deploy no `package.json`.

10. Pontos Criticos

Esta seccao identifica os pontos de maior atencao tecnica que devem ser considerados durante a implementacao do sistema de propostas avancadas.

10.1 Conectividade DB

Nao foi possivel estabelecer ligacao directa PostgreSQL a partir deste ambiente (erro ENETUNREACH por resolucao IPv6). A API REST do Supabase tambem nao respondeu com as credenciais fornecidas (`service_role` key invalida ou expirada). Isto significa que todas as operacoes DB devem ser executadas pelo utilizador via Supabase Dashboard, CLI, ou pela aplicacao em producao. As migrations SQL foram preparadas e validadas sintacticamente, mas a aplicacao remota depende de intervencao manual para ser aplicada.

10.2 Migration do Blueprint Engine

A migration 20260807000000 cria 5 tabelas novas, mas o seu estado de aplicacao no Supabase remoto e desconhecido. A migration e idempotente (CREATE IF NOT EXISTS) pelo que pode ser re-executada com seguranca. No entanto, a migration de seed data (20260807010000) que insere categorias e blueprints de exemplo usa INSERT sem ON CONFLICT, pelo que nao e idempotente e deve ser executada apenas uma vez.

10.3 Tipos Supabase Auto-Generated

O ficheiro src/integrations/supabase/types.ts e auto-gerado pelo Supabase CLI. Apos aplicar as migrations do Blueprint Engine, este ficheiro deve ser regenerado com supabase gen types para que os tipos TypeScript reflectam as novas tabelas e colunas. Sem esta regeneracao, o TypeScript reportara erros de tipos em todas as operacoes nas novas tabelas.

10.4 Separacao de Modulos

A separacao entre o fluxo simples e o fluxo avancado e bem definida ao nivel de paginas e servicos, mas existe partilha do mesmo Supabase client e do mesmo contexto de autenticao. O sistema avancado depende da organizacao activa (organization?.id) para criar propostas, mas nao valida explicitamente se a organizacao tem um plano que suporta propostas avancadas. Esta validacao deve ser adicionada antes da criacao de propostas avancadas.

11. Riscos

Risco	Severidade	Probabilidade	Mitigacao
Migration nao aplicada no Supabase remoto	Alta	Media	Executar via Dashboard SQL Editor antes de qualquer operacao
Tipos TypeScript desactualizados	Media	Alta	Executar supabase gen types apos migration
RLS polices em falta nas novas tabelas	Alta	Baixa	A migration inclui RLS para todas as novas tabelas; verificar com diag
Conexao IPv6 no ambiente actual	Media	Alta	Usar Supabase Dashboard ou CLI para operacoes DB; nao depende d
jsPDF limitacoes para PDFs premium	Media	Confirmada	Migracao aprovada para HTML/CSS + Playwright nos templates PRO
AI hallucination em conteudo gerado	Alta	Media	Sistema anti-hallucination: AI preenche dentro de estrutura aprovada,
Playwright bundle size impact	Baixa	Media	Lazy loading; Playwright e ~15MB gzip mas so carregado quando neco
Divergencia de dados entre propostas simples e avancadas	Media	Media	Manter separation of concerns; opcional: future sync via shared propos

12. Plano de Implementacao

O plano de implementacao segue a diretiva do Protocolo Rule 48: apos a auditoria BASELINE, prosseguir automaticamente para a implementacao sem pedir permissao. O plano esta organizado em fases sequenciais com checkpoints de verificacao.

Fase 1: Fundacao DB (Prioridade Maxima)

- Verificar/aplicar migration 20260807000000 no Supabase remoto
- Aplicar migration de seed data 20260807010000 (categorias + blueprints)
- Executar supabase gen types para regenerar tipos TypeScript
- Executar diagnostics.sql para verificar integridade do schema
- Verificar RLS policies nas 5 novas tabelas

Fase 2: Document Model + Question Engine

- Implementar DocumentModel que transforma answers + blueprint em estrutura de documento
- Criar type-safe content rules validation (minWords, maxWords, tone, etc.)
- Implementar visibility rules engine (showIf com equals/not_equals/contains)
- Adicionar validacao de respostas obrigatorias no wizard PreencherProposta.tsx

Fase 3: AI Integration (por seccao)

- Criar AI service que gera conteudo por seccao (nao documento inteiro)
- Implementar prompt engineering com contexto da seccao + respostas + info da empresa
- Sistema anti-hallucination: AI preenche dentro de estrutura aprovada
- Usar [INFORMACAO EM FALTA] para gaps de dados
- Streaming de geracao com indicador de progresso
- Guardar ai_content, ai_model, ai_tokens_used em proposal_section_answers

Fase 4: HTML/CSS Renderer (Premium PDFs)

- Criar design system base (typography scale, color system, spacing)
- Implementar render HTML a partir do DocumentModel
- Usar Playwright page.pdf() para output vectorial
- Criar pelo menos 1 template premium HTML/CSS
- Manter jsPDF para templates free (classic, modern, executive)

Fase 5: Brand Profiles + Design System

- UI para configurar Brand Profile (cores, fonte, estilo visual)
- Integrar Brand Profile com o HTML/CSS renderer
- Extracao automatica de cores do logo (se disponivel)

Fase 6: Revisao + Exportacao

- Pagina de revisao com todas as seccoes geradas
- Edicao inline do conteudo gerado pela AI
- Exportacao PDF (premium via Playwright, standard via jsPDF)
- Controlo de versoes do blueprint (blueprint_version)

Fase 7: Auditoria + Validacao Final

- Verificar RLS em todas as operacoes do novo sistema
- Testes manuais do fluxo completo (categoria -> blueprint -> respostas -> AI -> PDF)
- Executar diagnostics.sql completo
- Verificar que o fluxo simples continua intacto (zero regressao)

Regra de ouro: O fluxo de propostas simples (CriarProposta, ResumoProposta, PDF jsPDF) e sagrado. Nenhum codigo existente deve ser modificado sem necessidade absoluta. Todo o novo sistema de propostas avancadas e adicionado em paralelo, com rotas, servicos e tipos proprios.